

WRF auf GPUs

Gesamturteil

Stand Mai 2026 gibt es **keine offizielle, modern gepflegte GPU-Version von WRF-ARW**, die dem aktuellen CPU-WRF in Breite und Reife entspricht. Im offiziellen WRF/MPAS-Umfeld bezog sich die GPU-/OpenACC-Arbeit zuletzt auf **MPAS-Atmosphere**, nicht auf ein offizielles WRF-CUDA-Release; in einem offiziellen WRF-Support-Thread wurde 2019 ausdrücklich gesagt, dass WRF selbst damals keine offizielle GPU-Fähigkeit habe. Parallel dazu dokumentiert MPAS inzwischen einen GPU-fähigen **Dycore**, aber noch keine vollständigen GPU-Physics-Suites. ¹

Die veröffentlichte WRF-auf-GPU-Literatur ist außerdem **stark schief verteilt**: Sie zeigt überwiegend sehr große Beschleunigungen für **isolierte Microphysics- oder Radiation-Module**, aber fast nie eine belastbare, moderne **End-to-End-Portierung eines vollständigen WRF-v4-Operations-Stacks** mit Dycore, Physics, Halos, I/O, Diagnostik und Nests. Dem gegenüber stehen inzwischen **echte Vollsystem-Erfolge** bei Nachbarmodellen: MeteoSwiss fährt **ICON operativ auf GPUs** und berichtet für den Gesamtworkflow eine **5,5× Socket-zu-Socket-Beschleunigung**; MPAS ist offiziell auf dem Weg, hat aber derzeit nur den Dycore auf GPU. ²

Vor diesem Hintergrund ist das Scheitern deines lokalen OpenACC-Versuchs **nicht überraschend**, sondern inhaltlich sehr plausibel. Dein README beschreibt eine teilweise lauffähige GPU-Binärdatei, aber eine **launch-bound** Architektur mit viel GPU-Leerlauf und ungelösten Physics-Problemen. Genau dieses Muster passt zu dem, was die aktuelle Literatur für Wettermodelle nahelegt: **isolierte Offload-Schritte** bringen wenig, wenn der Zustand nicht dauerhaft auf dem Gerät bleibt und der Timestep nicht als Ganzes neu gedacht wird. ICON formuliert dieses Problem sehr klar: atmosphärische Modelle haben meist **niedrige arithmetic intensity**, sodass isolierte GPU-Kernel durch CPU-GPU-Datentransfers ausgebremst werden; deshalb wurde dort eine **Full-Port-Strategie** gewählt. ³

Mein kompaktes Urteil lautet deshalb: **Kein zweiter großer Versuch mit Legacy-WRF + inkrementellem OpenACC**. Wenn du es noch einmal angehst, dann als **Kern-Neuschreibung der Time-Stepping-Engine** mit residentem GPU-Zustand, GPU-aware Halos, asynchronem I/O und einer **formalen, nicht-bitweisen Verifikationskette**. Genau dort liegen heute die höchsten Erfolgschancen. ⁴

Was bisher publiziert oder sichtbar versucht wurde

Die erste wirklich bekannte WRF-GPU-Arbeit im NCAR-Umfeld ist **Michalakes und Vachharajani 2008**. Dort wurde ein „computationally intensive portion“ von WRF auf CUDA gebracht. Das Ergebnis war **nahezu 10×** für diesen Teil, aber nur **1,23× für das Gesamtmodell**. Diese eine Zahl ist in meinen Augen historisch extrem wichtig: Sie zeigt schon 2008, dass spektakuläre Kernel-Speedups **nicht automatisch** zu relevantem End-to-End-Gewinn werden. Das damalige NCAR-Benchmark-Programm zerlegte das Problem dann in einzelne WRF-Kerne wie **WSM5**, **Scalar Advection** und **Chemistry**; Longwave- und Shortwave-Radiation waren damals auf der Benchmark-Seite ausdrücklich noch als „GPU implementation needed“ markiert. ⁵

Danach folgte eine Reihe von **WRF-Physics-Einzelports**. Für **WSM5** wurde 2012 eine CUDA-C-Version publiziert, die den Rechenzeitanteil des Schemas drastisch reduzierte und je nach Messweise **206× mit I/O** bzw. **389× ohne I/O** auf einer GPU berichtete; mit vier GPUs wurden noch größere Modul-Speedups angegeben. Für **Kessler** wurde 2013 eine CUDA-Implementierung publiziert, die **70×** gegen die single-threaded CPU und **132×** auf vier GPUs meldete. Für **WSM6** erschien 2015 ein CUDA-C-Paper mit **216×** auf einer Tesla K40. Daneben gibt es eine kleinere 2013er Arbeit aus Korea zu **CUDA Fortran** für „major physics routines“ in WRF; sie belegt, dass der CUDA-Fortran-Pfad früh erprobt wurde, liefert aber in den verfügbaren Abstracts deutlich weniger belastbare Architekturinformation als die großen Journal-Papers. ⁶

Radiation wurde ebenfalls separat untersucht. Für **RRTMG_LW** gibt es eine WRF-bezogene GPU-Arbeit, die ausdrücklich die **Machbarkeit im WRF-Kontext** untersucht und in der zugänglichen Zusammenfassung sogar **bit-exakte** Outputs zur Original-Fortran-Version hervorhebt. Für **RRTMG_SW** gibt es eine publizierte CUDA-Arbeit; spätere Arbeiten beziehen sich auf diese Shortwave-Beschleunigung als etablierte Vorarbeit. Zusammen zeigen diese Beiträge: selbst sehr schwierige Physics sind **isoliert** GPU-fähig. Aber sie zeigen eben noch nicht, dass ein **aktuelles komplettes WRF** dadurch automatisch wirtschaftlich portierbar wird. ⁷

Neuere Arbeiten bestätigen dieses Muster. 2021 wurde im MPAS-Kontext eine **OpenACC-Portierung der WRF-WSM6-Microphysics** veröffentlicht. Sie meldet im Vollmodell-Setup im Mittel **2,38× gegen 48 MPI-CPU-Prozesse** und legt großen Wert auf **Verifikation**, inklusive Vergleich der CPU/GPU-Differenzen zu den Differenzen zwischen CPU-Compilern. 2024 erschien eine Arbeit zu **WSM7** auf heterogenen CPU/GPU-Systemen, die das Schema **in C++/CUDA neu implementiert** und **dynamisches Load Balancing** pro Zeitschritt verwendet; dort werden bis zu **28,51×** gegen die Referenzimplementierung des Schemas gemeldet. Diese Arbeiten sind ernst zu nehmen, aber wieder gilt: Es sind **Schema-Portierungen**, keine belastbaren Belege für einen modernen WRF-v4-End-to-End-Port. ⁸

Der offizielle Ökosystem-Status spricht dieselbe Sprache. Im WRF/MPAS-Supportforum ist die GPU/OpenACC-Ecke praktisch **MPAS-zentriert**. Die MPAS-Dokumentation sagt seit v8.3.0 klar, dass GPU-Ausführung verfügbar ist, aber **nur für den Dynamical Core**, nicht für die Physics-Suites; zusätzlich wurden in v8.4.0 **GPU-aware Halo Exchanges** und größere Datentransfer-Optimierungen eingeführt. Das ist wichtig, weil es indirekt zeigt, wo der Community-Fokus tatsächlich liegt: nicht bei einem späten Legacy-WRF-GPU-Port, sondern bei **neueren Kernen und saubererem Datenfluss**. ⁹

Was diese Literatur tatsächlich zeigt

Die wichtigste Erkenntnis lautet: **Die meisten spektakulären WRF-GPU-Speedups sind methodisch nicht mit deinem Operationsziel vergleichbar**. Sie vergleichen meist einen **einzelnen Physics-Kernel** gegen eine **single-core** oder kleine CPU-Referenz. Das ist wissenschaftlich legitim, aber für eine reale WRF-Produktionskette nur begrenzt aussagekräftig. Schon die frühe NCAR-Arbeit zeigte das offen: fast 10× in einem WRF-Teil ergaben nur 1,23× für das Gesamtmodell. ¹⁰

Die zweite Erkenntnis ist, dass **isolierte Offloads in Wettermodellen architektonisch oft falsch** sind. ICON beschreibt explizit, dass atmosphärische Codes typischerweise **niedrige arithmetic intensity** haben. Dort bestätigt die Laufzeitmessung sogar, dass das Kopieren des gesamten Zustands CPU→GPU ungefähr so teuer sein kann wie **ein kompletter GPU-Timestep**. Daraus folgt fast zwangsläufig: Wenn man wirklich gewinnen will, muss der **gesamte hochfrequentierte Timestep-Zustand resident auf der GPU** bleiben; CPU-GPU-Transfers dürfen im Zeitloop im Wesentlichen nur noch für **seltene Dinge wie Output** oder bewusst hybrid belassene Komponenten anfallen. ICON tut genau das. MPAS geht in dieselbe Richtung, indem es Datentransfers größer bündelt und GPU-aware Halos einführt. ¹¹

Die dritte Erkenntnis ist differenzierter: **OpenACC ist nicht per se tot**, aber Legacy-OpenACC ist auch **kein Wundermittel**. MeteoSwiss hat mit ICON gezeigt, dass man mit OpenACC einen vollständigen NWP-Workflow bis in den operativen Betrieb bringen kann. Gleichzeitig macht dieselbe Arbeit deutlich, wie groß der Aufwand ist: ungefähr **21 000 OpenACC-Pragmas** in einer sehr großen Fortran-Codebasis, eine konsequente **Full-Port-Strategie**, spezielle Datenflussentscheidungen, probabilistische Tests, mehrsaisonale Verifikation und fortlaufende Community-Schulung. Außerdem nennt die Autorenenarbeit selbst die Grenzen: volle Unterstützung im Wesentlichen bei **NVHPC auf NVIDIA**, deutlich schlechtere Portabilität auf AMD/Intel, sinkende Lesbarkeit durch die Überlagerung von OpenACC, OpenMP und vendorspezifischen Optimierungsdirektiven. Für ein altes, stark verzweigtes WRF ist das kein attraktiver Startpunkt mehr. ¹²

Die vierte Erkenntnis ist die eigentlich entscheidende: **Erfolgreiche GPU-Atmosphärenmodelle der neueren Generation setzen architektonisch auf Neuaufbau oder tiefen Refactor, nicht auf bloßes Offload-Patching.**

HOMMEXX hat einen Fortran-Dycore in **C++/Kokkos** neu implementiert und damit Performance-Portabilität auf CPUs, KNL und NVIDIA-GPUs demonstriert. SCREAM wurde **von Grund auf in C++ mit Kokkos** für CPUs und GPUs entworfen und ist nach mehreren Jahren Entwicklung produktionsreif genug für exascale-nahe Einsätze. Pace zeigt, dass ein **GT4Py-basiertes DSL** für FV3-Dycore plus Cloud Microphysics auf GPU-Beschleunigern **3,5-4× schneller** als die Fortran-Referenz laufen kann und dabei die Wartbarkeit verbessert. Das 2026er **icon-exclaim**-Paper zeigt sogar, dass ein **GT4Py-refaktorierte ICON-Dycore** die bestehende OpenACC-Dycore-Version leicht **übertreffen** kann und dabei mit einem mehrstufigen Testregime entwickelt wurde. Die Met-Office-Arbeit zu **LFRic** formuliert das zugrunde liegende Prinzip am klarsten: **separation of concerns** zwischen Wissenschaftscode und Parallelisierungscode, plus DSL-gestützte Codegenerierung. ¹³

Genau deshalb ist dein README in seiner Grundintuition richtig: **Es braucht sehr wahrscheinlich einen Kern-Neubau**. Wo ich allerdings leicht von der README-Empfehlung abweiche, ist die strategische Wahl der Ausführungsschicht. Ein **NVIDIA-only CUDA-Fortran-Neubau** kann taktisch sinnvoll sein; die stärkste Evidenz für **langfristig robuste** Atmosphären-Neuarchitekturen kommt heute aber eher aus **Kokkos-/C++- und DSL-basierten** Ansätzen. ¹⁴

Welche Kernarchitektur ich heute wählen würde

Wenn du mich nach der **höchsten Wahrscheinlichkeit für einen langfristig erfolgreichen zweiten Versuch** fragst, würde ich **nicht** die bestehende WRF-v4-Fortran-Basis in OpenACC weiterverfolgen. Ich würde auch **nicht** als erstes Ziel versuchen, „ganz WRF“ in einem Zug neu zu schreiben. Stattdessen würde ich eine **neue GPU-native ARW-kompatible Time-Stepping-Engine** bauen, mit klarer Trennung zwischen **Ausführungsmodell, Dycore/Transport** und **Physics-Kernel-API**. Diese Trennung ist genau das Muster, das in LFRic, HOMMEXX, SCREAM, Pace und icon-exclaim wiederkehrt. ¹⁵

Meine strategische Empfehlung wäre dabei: **Kokkos/C++ als primäre Laufzeit- und Datenmodell-Schicht**, optional ergänzt durch eine **stencil-orientierte DSL** für die wirklich regulären Dycore-Teile. Der Grund ist einfach. WRF besteht nicht nur aus schönen regulären 3D-Stencils. Es besteht auch aus **column physics**, vielen Spezialfällen, unfreundlicher Datenhistorie, gelegentlich irregulären Mustern, MPI-Grenzen, Diagnostik, Nests und I/O. Reine Stencil-DSLs sind dafür fantastisch im Dycore, aber **nicht automatisch** die beste universelle Antwort auf den gesamten Physics-Stapel. Kokkos-basierte Projekte haben genau diese Breite schon bewiesen; DSLs wie GT4Py zeigen dagegen einen sehr überzeugenden Weg für die **stencil-lastigen Kernteile**. ¹⁶

CUDA Fortran würde ich nicht verwerfen. Im Gegenteil: Wenn dein Ziel ausdrücklich **NVIDIA-only** ist, NVHPC gesetzt ist und du möglichst viel der Fortran-Denkweise erhalten willst, dann ist CUDA Fortran die **beste taktische Alternative**. Die NVIDIA-Dokumentation zeigt sauber, dass man damit explizite Kernel, Device-Routinen, Launch-Bounds und Speicherattribute auf Fortran-Ebene kontrollieren kann. Das ist für ein schnelles, hart optimiertes NVIDIA-System attraktiv. Strategisch bleibt es aber vendor-gebundener als Kokkos oder eine moderne DSL, und genau diese Vendor-Bindung ist etwas, worunter OpenACC-Ports heute sichtbar leiden. Deshalb wäre CUDA Fortran für mich **Plan B für ein NVIDIA-spezifisches Produkt**, nicht die bevorzugte Endarchitektur eines mehrjährigen Rewrites. ¹⁷

Unabhängig von der Programmiersprache würde ich die Kernarchitektur nach sechs Prinzipien bauen:

Erstens: **Whole-state device residency**. Nach der Initialisierung lebt der prognostische Zustand auf der GPU; Transfers zur CPU passieren nur für Output, seltene Hybrid-Komponenten oder Debugging. ICON begründet dieses Prinzip direkt, und MPAS bewegt sich mit seinen gebündelten Transfers und GPU-aware Halos in dieselbe Richtung. ¹⁸

Zweitens: **ein MPI-Rank pro GPU mit GPU-aware Halo Exchange**. Das ist heute der Standardpfad in MPAS und ein praktisch unvermeidlicher Baustein für reale Skalierung. ¹⁹

Drittens: **ein explizites Block-/Tile-/Column-Datenmodell**. WSM7 zeigt, dass vertikale Spalten eine natürliche Parallelisierungseinheit für Microphysics sind; ICON ordnet seine Daten blockbasiert an; GT4Py und Kokkos gewinnen sehr viel aus expliziter Datenlayout-Kontrolle statt aus impliziter Compiler-Magie. ²⁰

Viertens: **Fusion auf Timestep-Ebene** statt Mikro-Kernel-Orgie. icon-exclaim sagt explizit, dass die langfristige Optimierung in der **Fusion größerer Stencils** liegt, um Datenwiederverwendung auszunutzen. Genau das passt auch zu deinem lokalen Befund, dass zu viele kleine Launches das System ausbremsen. ²¹

Fünftens: **asynchrones I/O und GPU-seitige Diagnostik**, sodass nur fertige Felder an Output-Zeitpunkten in den Hostraum wandern. Genau so begründet es ICON für seine hochfrequenten Diagnostiken. ²²

Sechstens: **explizite Präzisionspolitik**. Nicht „irgendwann vielleicht Mixed Precision“, sondern ein von Anfang an dokumentiertes Feld-für-Feld-Konzept mit FP64-Referenzpfad, später gezieltem Übergang auf FP32 und nur dort weiter herunter, wo Verifikation es erlaubt. ICON berichtet validierte Mixed-Precision im Dycore ohne Verschlechterung der meteorologischen Scores, aber eben nur nach langer Prüfstrecke. ²³

Wenn du eine wirklich harte Entscheidung willst, dann ist meine Antwort also:

Strategisches Ziel:

eine **GPU-native, resident-data, performance-portable Kern-Neuschreibung** des ARW-Zeitsteppers, **primär auf Kokkos/C++**, optionally mit einer **GT4Py-artigen DSL für stencil-lastige Dycore-Teile**.

Taktischer NVIDIA-Shortcut:

falls du exklusiv NVIDIA verfolgst und der erste operative Prototyp wichtiger ist als Portabilität, dann ist **CUDA Fortran** die beste Ausweichroute – aber bitte mit **genau derselben resident-data- und Verifikationsarchitektur**, nicht als „OpenACC plus ein paar Kernels“. ²⁴

Wie Verifikation ohne Bitidentität praktisch funktionieren sollte

Die kurze Antwort auf deine Schlüsselfrage lautet: **Ja, es gibt Vorbilder – aber kein einziges fertiges WRF-spezifisches Komplettpaket**. Was es gibt, sind mehrere starke Bausteine, die man zu einer **mehrschichtigen Verifikationspyramide** kombinieren kann. ICONs **proptest**, die **TSC1.0**-Methode, E3SMs **EVV4ESM**-Ansatz und die **operativen Forecast-Verification-Frameworks** wie METplus ergeben zusammen fast genau das, was du beschreibst. ²⁵

Die unterste Ebene ist ein **temporärer Identitätsmodus für kleine Debug-Fälle**. MPAS dokumentiert dafür konkret die NVHPC-Optionen `-Mnofma` und `-gpu=math_uniform`, mit denen GPU- und CPU-Runs unter bestimmten Bedingungen bitweise identisch gemacht werden können. Die Doku sagt gleichzeitig ausdrücklich, dass das **nicht für Produktion** gedacht ist, weil es Performance kostet. Das ist genau richtig eingeordnet: nicht als wissenschaftliches Ziel, sondern als **kurzfristiges Debug-Werkzeug**, um grobe Fehler sehr früh zu isolieren. ²⁶

Die nächste Ebene ist die wichtigste für einen Rewrite: **Savepoint- bzw. Substitutions-Tests an Modulgrenzen**. Das beste veröffentlichte Muster dafür ist meiner Ansicht nach **icon-exclaim**. Dort wurde mit einem Preprocessor ein **verification mode** und ein **substitution mode** aufgebaut: einer führt Referenz- und neue Implementierung parallel aus und vergleicht sie mit Toleranzen; der andere ersetzt den alten Codepfad durch den neuen, sobald der Vergleich stabil ist. Genau so würde ich es für einen WRF-Nachfolger bauen: an klar definierten Punkten im Timestep werden Inputs/Outputs des Legacy-Codes erfasst, und der neue Kernel muss dort reproduzierbar innerhalb definierter Toleranzen bleiben. Für Debuggierbarkeit ist das Gold wert. ²⁷

Darüber kommt eine **probabilistische Toleranzprüfung** im Stil von **proptest**. Die Idee ist elegant: Man erzeugt auf der CPU eine kleine Ensemble-Streuung durch winzige Perturbationen in den letzten Bits der Eingänge und leitet daraus **variable-spezifische Toleranzen** ab. Dann akzeptiert man einen GPU-Lauf, wenn er **innerhalb dieser ensemble-basierten Streuung** liegt. ICON nutzt genau das in seiner CI-Pipeline; jede Änderung muss diese Tests auf reduzierten, aber codepfad-repräsentativen Konfigurationen bestehen. Für deine Frage nach „physical correctness“ statt Bitidentität ist das der **modernste veröffentlichte Ansatz**, den ich derzeit sehe. Außerdem ist proptest öffentlich als MeteoSwiss-Projekt sichtbar. ²⁸

Noch eine Ebene höher würde ich **TSC1.0** einsetzen. Diese Methode testet nicht, ob zwei Lösungen identisch sind, sondern ob eine Änderung **größer ist als die bekannte Zeitschritt-Sensitivität** des Referenzmodells. Das ist besonders passend, wenn du – wie du selbst vermutest – vielleicht später einen **anderen Timestep oder eine andere Parallelisierungsarchitektur** wählst. TSC ist ausdrücklich für geophysikalische Evolutionsgleichungen gedacht, billig genug für häufige Runs und gerade für nicht-bitweise, aber numerisch „harmlos“ geänderte Codes sehr wertvoll. ²⁹

Für längere Horizonte würde ich zusätzlich **EVV4ESM-artige Klima-/Statistik-Sameness-Tests** übernehmen. In E3SM wird zwischen drei Dingen getrennt: bit-identische Änderungen, **nicht-bit-identische aber klimatologisch gleiche** Änderungen und **klimatisch verändernde** Änderungen. Dazu werden Werkzeuge wie **MVK**, **PGN** und **TSC** genutzt, zusammen mit EVV-Auswertungen. Das ist nicht WRF-spezifisch, aber die Logik ist universell: Nicht nur einzelne Felder vergleichen, sondern **statistische Eigenschaften eines Kurzensembles**. ³⁰

Die äußerste Schicht ist dann die **echte meteorologische Verifikation** gegen Beobachtungen und Analysen. Hier gibt es bereits ausgereifte Werkzeuge. **MET/METplus** ist heute ein zentraler NOAA-/UFS-Stack für Verifikation, Validation und Diagnostik und deckt klassische Scores, räumliche Verifikation und

objektbasierte Methoden ab. Dazu kommen organisationsspezifische Regeln wie NCEP **EVS**. Für ein WRF-GPU-Rewrite würdest du genau solche Werkzeuge benutzen, um nicht nur Modell-gegen-Modell, sondern **Prognose-gegen-Realität** zu prüfen: Temperatur, Wind, Niederschlag, BIAS, RMSE, neighbourhood scores, objektbasierte Niederschlagsverifikation, Radar-/Satellitenvergleich und so weiter. ³¹

Parallel dazu würde ich eine **WRF-spezifische Testbank** aus idealisierten Fällen aufbauen. Die WRF-Dokumentation nennt selbst Fälle wie **em_hill2d_x**, **em_squall2d_x/y**, **em_b_wave**, **em_tropical_cyclone**, **em_les** und **em_seabreeze** als kontrollierte, architekturfreundliche Tests. Diese Fälle sind ideal, um numerische Eigenschaften wie Symmetrie, Wellenausbreitung, Topografieantwort, Turbulenz und konvektive Struktur zu prüfen, bevor man sich in Vollrealdaten verheddert. ³²

Schließlich braucht der neue Code dieselben klassischen **Software- und GPU-Korrektheitsprüfungen** wie jedes ernsthafte HPC-Projekt: **NaN/Inf-Wächter**, Positivität von Tracern, Massen- und Wasserbudgets, Restart-Konsistenz, Invarianz gegenüber Domain-Decomposition innerhalb tolerierter Grenzen sowie GPU-Debug-Tools. Für CUDA-seitige Laufzeitfehler sind **Compute Sanitizer** mit **memcheck**, **racecheck**, **initcheck** und **synccheck** heute Standard. Für Performance-Pathologien wie Registerdruck und Occupancy ist **Nsight Compute** mit Occupancy-Calculator direkt relevant. ³³

Wenn ich daraus eine konkrete Antwort auf deine Frage „hat jemand so etwas schon geschrieben?“ formuliere, dann lautet sie:

Ein einziges WRF-spezifisches Universalwerkzeug für „physical correctness“ existiert nach meinem jetzigen Recherchebild nicht.

Aber die Blaupause existiert bereits in Bausteinen:

- „**Substitution/Savepoint**“-Vergleich wie bei icon-exclaim. ²⁷
- **Probabilistische Toleranzen** wie bei ICON/probtest. ²⁸
- **Timestep-Convergence** wie TSC1.0. ²⁹
- **Ensemble-/Climate-Sameness** wie EVV4ESM. ³⁴
- **Forecast-vs-observation-Verifikation** mit METplus/EVS. ³⁵

Genau aus diesen fünf Teilen würde ich die Verifikationsplattform des Rewrites bauen.

Ein Umsetzungsfahrplan mit der höchsten Erfolgswahrscheinlichkeit

Der Weg mit der höchsten Erfolgswahrscheinlichkeit ist **nicht**, sofort Thompson + MYNN + Noah-MP + RRTMG + Nests + voller Canary-Workflow gleichzeitig auf GPU „irgendwie“ zu aktivieren. Der Weg mit der höchsten Erfolgswahrscheinlichkeit ist ein **gestaffelter Nachbau der Ausführungsebene**, bei dem die Verifikation von Anfang an Bestandteil der Architektur ist – so wie es ICON mit probtest und icon-exclaim mit Verification/Substitution gemacht haben. ³⁶

Ich würde den ersten Scope brutal klein halten: **ein Domain-Setup, ein Zeitschrittpfad, ein minimales Set an Physics**, kein Chemiepfad, keine exotischen Optionen, keine Nests in Phase eins. Dazu würde ich eine **referenzierbare CPU-Baseline** aufbauen und an mehreren Stellen im Timestep **Savepoints** definieren. Erst wenn Dycore, Advektion, Druck-/Metrikpfade und eine erste Physics-Kette stabil durch die Verifikationspyramide laufen, würde ich die nächste Physics-Suite hinzufügen. Das klingt langsamer, ist aber in Wahrheit schneller, weil du Fehler dann an **einem** Umbruchpunkt statt im Gesamtsystem suchst. Diese Denkweise ist konsistent mit den veröffentlichten Refactor-Vorbildern. ³⁷

Die Reihenfolge würde ich so wählen:

Zuerst **Dycore + Transport + Halo-Kommunikation**. Genau hier liegen die resident-data-, kernel-fusion- und Datenlayout-Fragen, die über den gesamten Erfolg entscheiden. Die offiziellen MPAS-GPU-Dokumente und ICON zeigen, wie wichtig Datentransfer-Minimierung und GPU-aware Halos dafür sind.

38

Danach **eine einzige repräsentative Physics-Kette** aus deinem Operationsbetrieb. Nicht die einfachste im Lehrbuch, sondern die, die du wirklich brauchst – aber möglichst isoliert genug, um die Verifikation nicht zu sprengen. Wenn Thompson/MYNN/RRTMG/Noah-MP der reale Zielstack ist, würde ich dennoch nicht mit allen vier gleichzeitig starten. Praktisch sinnvoll wäre meist: zuerst **Microphysics**, dann **PBL/Surface**, dann **Radiation**, dann kompliziertere Land-/Coupler-Themen. Dass Radiation separat behandelbar ist, zeigen die publizierten RRTMG-Arbeiten; dass Microphysics spaltenweise gut parallelisierbar ist, zeigen praktisch alle WSM/Kessler/Thompson-nahen Arbeiten.

39

Nesting würde ich bewusst spätestmöglich einführen. WRF-Nests sind nicht bloß „mehr vom Gleichen“, sondern bringen weitere zeitliche Kopplung, Randbedingungslogik, Interpolation und potenziell asynchrone Pfade hinein. Für die erste Erfolgsdefinition würde ich daher auf **eine saubere Single-Domain- oder Single-Resolution-Version** zielen. Erst wenn diese stabil, schnell und korrekt ist, kommt Multi-Nest. Dass diese Art von Stufung sinnvoll ist, folgt direkt aus der Literatur: die erfolgreichen GPU-Projekte sichern zuerst den Kern und erweitern dann systematisch.

40

Wenn du das Projekt so aufsetzt, ist der richtige Erfolgsmaßstab nicht „wir haben einen GPU-Build“. Der richtige Maßstab wäre:

- residenter GPU-Zustand ohne hochfrequente Host-Device-Kopien im Timestep, 18
- stabile Savepoint-/Substitution-Tests gegen die CPU-Referenz, 21
- probabilistische CI-Toleranztests, 22
- TSC/ensemble-basierte Nicht-BFB-Tests, 41
- meteorologische Verifikation gegen Beobachtungen/Analysen, 35
- und erst **danach** aggressive Mixed Precision und weitere Physics-Komplexität. 42

Wenn du diese Reihenfolge diszipliniert einhältst, wird der zweite Versuch nicht „sicher“ im mathematischen Sinn – Forschung ist nie sicher. Aber er wird von einem **riskanten Compiler-Offload-Projekt** zu einem **engineered system rewrite** mit deutlich besserer Erfolgswahrscheinlichkeit.

Offene Fragen und Grenzen

Diese Recherche ist breit, aber nicht allwissend. Ein Teil der WRF-GPU-Historie lebt in **älteren Konferenzbeiträgen, Thesen, Workshop-Folien und schwer zugänglichen PDFs**; dort kann es noch weitere Teilversuche geben, die in den großen Suchindizes nur unvollständig auftauchen. Einige ältere Abstracts sind außerdem besser zugänglich als die Volltexte, weshalb ich bei ein paar Spezialarbeiten bewusst **keine zu genauen Detailzahlen** wiederholt habe, wenn die zugängliche Quelle dafür nicht sauber genug war.

43

Außerdem habe ich für die lokale Projektanalyse **nur dein bereitgestelltes README** verwendet, nicht die erwähnten internen Entscheidungspapiere und Sprint-Historien. Es ist gut möglich, dass diese internen Dokumente noch weitere harte Evidenz enthalten, die meine Architekturpräferenz weiter schärfen würde – insbesondere zur Verteilung der Zeit auf Dycore, Microphysics, Radiation, Noah-MP, Nesting und I/O.

Trotz dieser Grenzen ist das Gesamtbild aus meiner Sicht stabil:

Nicht weiterpatchen. Resident-data-Core neu bauen. Verification first. OpenACC nicht als Hauptstrategie. Kokkos/modernes Kernel-Layer als strategisches Ziel, CUDA Fortran nur als bewusst gewählte NVIDIA-only-Abkürzung. 44

1 9 <https://forum.mmm.ucar.edu/forums/gpu-openacc.128/>
<https://forum.mmm.ucar.edu/forums/gpu-openacc.128/>

2 3 4 11 12 18 22 23 25 28 36 40 42 44 <https://gmd.copernicus.org/articles/19/755/2026/>
<https://gmd.copernicus.org/articles/19/755/2026/>

5 10 <https://impacts.ucar.edu/en/publications/gpu-acceleration-of-numerical-weather-prediction-2>
<https://impacts.ucar.edu/en/publications/gpu-acceleration-of-numerical-weather-prediction-2>

6 https://www.researchgate.net/publication/260635902_Improved_GPUCUDA_based_parallel_weather_and_research_forecast_WRF_Single_Moment_5-class_WSM5_cloud_microphysics
https://www.researchgate.net/publication/260635902_Improved_GPUCUDA_based_parallel_weather_and_research_forecast_WRF_Single_Moment_5-class_WSM5_cloud_microphysics

7 39 https://www.researchgate.net/publication/273395467_GPU-Accelerated_Longwave_Radiation_Scheme_of_the_Rapid_Radiative_Transfer_Model_for_General_Circulation_Models_RRTMG
https://www.researchgate.net/publication/273395467_GPU-Accelerated_Longwave_Radiation_Scheme_of_the_Rapid_Radiative_Transfer_Model_for_General_Circulation_Models_RRTMG

8 <https://www.sciencedirect.com/science/article/pii/S0098300420306051>
<https://www.sciencedirect.com/science/article/pii/S0098300420306051>

13 16 24 <https://gmd.copernicus.org/articles/12/1423/2019/>
<https://gmd.copernicus.org/articles/12/1423/2019/>

14 17 <https://docs.nvidia.com/hpc-sdk/compiler/cuda-fortran-prog-guide/index.html>
<https://docs.nvidia.com/hpc-sdk/compiler/cuda-fortran-prog-guide/index.html>

15 <https://www.sciencedirect.com/science/article/pii/S0743731518305306>
<https://www.sciencedirect.com/science/article/pii/S0743731518305306>

19 26 38 https://www2.mmm.ucar.edu/projects/mpas/site/documentation/users_guide/model_options.html
https://www2.mmm.ucar.edu/projects/mpas/site/documentation/users_guide/model_options.html

20 <https://link.springer.com/article/10.1007/s11227-024-06009-9>
<https://link.springer.com/article/10.1007/s11227-024-06009-9>

21 27 37 <https://gmd.copernicus.org/articles/19/713/2026/gmd-19-713-2026.html>
<https://gmd.copernicus.org/articles/19/713/2026/gmd-19-713-2026.html>

29 41 <https://gmd.copernicus.org/articles/10/537/2017/gmd-10-537-2017.html>
<https://gmd.copernicus.org/articles/10/537/2017/gmd-10-537-2017.html>

30 34 <https://e3sm.atlassian.net/wiki/spaces/DOC/pages/1028719501/Climate%2Breproducibility%2Bnon-bit-for-bit%2Btesting>
<https://e3sm.atlassian.net/wiki/spaces/DOC/pages/1028719501/Climate%2Breproducibility%2Bnon-bit-for-bit%2Btesting>

31 35 <https://metplus.readthedocs.io/>
<https://metplus.readthedocs.io/>

32 https://www2.mmm.ucar.edu/wrf/site/online_tutorial/ideal_exercise.html

https://www2.mmm.ucar.edu/wrf/site/online_tutorial/ideal_exercise.html

33 <https://docs.nvidia.com/compute-sanitizer/ComputeSanitizer/index.html>

<https://docs.nvidia.com/compute-sanitizer/ComputeSanitizer/index.html>

43 <https://www.researchgate.net/publication/>

279156243_GPU_Compute_Unified_Device_Architecture_CUDA-

based_Parallelization_of_the_RRTMG_Shortwave_Rapid_Radiative_Transfer_Model

[https://www.researchgate.net/publication/279156243_GPU_Compute_Unified_Device_Architecture_CUDA-](https://www.researchgate.net/publication/279156243_GPU_Compute_Unified_Device_Architecture_CUDA-based_Parallelization_of_the_RRTMG_Shortwave_Rapid_Radiative_Transfer_Model)

[based_Parallelization_of_the_RRTMG_Shortwave_Rapid_Radiative_Transfer_Model](https://www.researchgate.net/publication/279156243_GPU_Compute_Unified_Device_Architecture_CUDA-based_Parallelization_of_the_RRTMG_Shortwave_Rapid_Radiative_Transfer_Model)